

Время: 00:00–06:00

ЭКРАН: доска, зона «Что было в первой части» → карта текущего выпуска → текстовая карточка DEVOPS MAY CRY.

ГОВОРЮ

«В первом видео кластер уже был готов. На нём мы разобрали архитектуру Kubernetes, основные объекты и посмотрели, что происходит при `kubect1 apply`. Но я не показывал, откуда берутся сами машины, кто ставит на них Kubernetes и чем self-managed кластер отличается от managed. Путь запроса от браузера до Pod'a тоже остался на продолжение — к нему мы придём в четвёртой части этого выпуска.

Сегодня закрываем именно пробел вокруг платформы. Начнём с маленьких примеров Terraform и Ansible, затем посмотрим исходники Kubespray, создадим инфраструктуру кодом, соберём self-managed кластер, рядом поднимем managed и сравним их не по рекламной таблице, а по тому, что реально видно в терминале.

Для лабораторий у нас одно сквозное приложение — DEVOPS MAY CRY. Это вымышленная ночная диспетчерская: принимает заявки о странных инцидентах, хранит их в Postgres и умеет создавать нагрузку для следующих частей. Название — отсылка к Devil May Cry; история диспетчерской и техническая модель придуманы для курса. Технически это обычный Go-сервис с Dockerfile, Compose, probes, ресурсами и отдельными Kubernetes-манифестами. Поэтому taints, OOM, HPA, DNS и внешний вход дальше проверяем на одном приложении, а не на сорока случайных nginx.

Сейчас ничего не запускаю вслепую. Сначала фиксирую корень клона и локальные параметры записи. Секреты и реальные inventory в Git не попадают».

🖥️ ПОКАЗАТЬ

- Корень чистого клона, а не папку подготовки видео.
- `apps/devops-may-cry`: Go-код, Dockerfile, Compose, migrations.
- `kubernetes/devops-may-cry`: base, scheduling, scaling, services.
- `recording/recording.env.example`: только названия переменных; реальный `.recording.env` не раскрывать.

📋 V2-C1-S01-C01 · Зафиксировать корень клона и параметры записи

```
REPO_ROOT="$(git rev-parse --show-toplevel)"
cd "$REPO_ROOT"
test -f recording/.recording.env || cp recording/recording.env.example
recording/.recording.env
set -a
source recording/.recording.env
set +a
printf 'repo=%s\nself=%s\nmanaged=%s\n' "$REPO_ROOT" "$SELF_CONTEXT"
"$MANAGED_CONTEXT"
```

✅ После команды

Ожидаю: абсолютный путь текущего клона и контексты `kubespray` / `yc-managed`. Если env-файл только что создан, остановись до REC и заполни его.

🧠 ЧТО ЭТО ЗНАЧИТ

Репозиторий переносим между машинами без абсолютных путей. Публичный код хранит только примеры и структуру; локальные адреса, ключи, inventory и state остаются вне Git.

⚠ ЕСЛИ НЕ СОВПАЛО

Если `git rev-parse` не сработал — открыт не клон репозитория. Если env-файл только создан — стоп, заполнить вне кадра и заново начать сцену.

ПЕРЕХОД

«Сначала два маленьких примера. После них в большом Kubespray-проекте будут знакомы inventory, playbook, role и идемпотентность».

с1.1 Terraform: код описывает желаемое состояние

Время: 06:00–17:00

ЭКРАН: VS Code, `01_ЧАСТЬ_1_КЛАСТЕР/01_1.1_hello_terraform`.

ГОВОРЮ

«Terraform не исполняет список шагов сверху вниз как shell-скрипт. Он читает конфигурацию, строит граф зависимостей, сравнивает желаемое состояние с тем, что знает из state, и предлагает план.

Открываю `main.tf`. Здесь специально нет облака: провайдер `local` создаёт два обычных файла — журнал и сводку смены. Так можно увидеть весь цикл без ожидания VM и без счёта за ресурсы.

`terraform init` загружает провайдер и создаёт lock-файл. В боевом репозитории lock-файл коммитят: он фиксирует выбранную версию провайдера. `plan` пока ничего не меняет. Он показывает две операции create».

ПОКАЗАТЬ

- `terraform { required_providers }`.
- `variable "dispatcher"`.
- Два `local_file` и зависимость сводки от журнала.
- `output`, не раскрывая никакие секреты.

V2-C1-S02-C01 · Terraform init и plan на двух обычных файлах

```
cd "$REPO_ROOT/01_ЧАСТЬ_1_КЛАСТЕР/01_1.1_hello_terraform"
rm -f zhurnal.txt svodka.txt terraform.tfstate terraform.tfstate.backup
terraform init
terraform plan
```

✓ После команды

Ожидая: `Plan: 2 to add, 0 to change, 0 to destroy`. В кадре показываем `main.tf` до запуска.

🗣 ГОВОРЮ

«Теперь применяю план. На интерактивном подтверждении Terraform принимает полное `yes`. После `apply` сверяю не только зелёную строку, но и реальные файлы. Повторный `plan` должен быть пустым — это базовая проверка, что код и состояние совпадают».

📄 V2-C1-S02-C02 · Terraform apply, state и идемпотентность

```
terraform apply
cat zhurnal.txt
cat svodka.txt
terraform plan
terraform output
```

✓ После команды

Ожидая: после `yes` создаются два файла; повторный `plan` сообщает `No changes`.

🗣 ГОВОРЮ

«Теперь имитирую ручную правку мимо кода. Меняю журнал, запускаю `plan` — Terraform видит расхождение и предлагает вернуть ресурс к описанному состоянию. Это `drift`. В реальном проекте такой `plan` часто первым показывает, что кто-то подкрутил облако руками.

Важный нюанс: Terraform не всеведущий. Он сравнивает код, state и данные провайдера. Потерянный или общий локальный state — риск. В команде state обычно лежит в удалённом backend с блокировкой и контролем доступа».

☰ V2-C1-S02-C03 · Дрифт, восстановление из кода и cleanup

```
echo "тут никого не было" > zhurnal.txt
terraform plan
terraform apply
head -2 zhurnal.txt
terraform destroy
```

✅ После команды

Ожидая: Terraform видит ручную правку как расхождение, восстанавливает объявленное содержимое, затем удаляет свои два ресурса.

🧠 ЧТО ЭТО ЗНАЧИТ

`resource` описывает объект, `variable` — вход, `output` — интерфейс наружу, `state` — связь между кодом и реальными объектами. `plan` показывает намерение, `apply` меняет среду.

⚠️ НЕ ГОВОРИТЬ

Не называть state «единственным источником истины»: желаемое состояние — код, соответствие реальным объектам Terraform отслеживает через state и refresh провайдера.

ПЕРЕХОД

«Terraform создаёт инфраструктуру. Но внутри новой Linux-машины Kubernetes сам не появится. Для настройки машин нужен другой слой».

c1.2 Ansible: привести машины к нужному состоянию

Время: 17:00–29:00

ЭКРАН: 01_ЧАСТЬ_1_КЛАСТЕР/02_1.2_hello_ansible.

ГОВОРЮ

«Ansible берёт inventory, подключается к хостам и выполняет модули. В обычном Linux-сценарии агент на сервер ставить не надо: достаточно SSH и Python. Здесь первый пример локальный, поэтому в inventory явно написано

`ansible_connection=local`. Это учебная замена SSH, а не удалённый сервер.

Открываю `playbook.yml`. `hosts` выбирает группу, `vars_files` подключает тарифы, `tasks` описывают состояние файлов, а handler запускается только когда задача сообщила `changed`.

Сначала очищаю результат прошлого дубля, затем проверяю inventory и запускаю playbook».

ПОКАЗАТЬ

- `inventory.ini` и локальный тип соединения.
- `vars/prices.yml`.
- `playbook.yml`: template/copy/file, notify и handlers.

V2-C1-S03-C01 · Ansible inventory, ping и первый play

```
cd "$REPO_ROOT/01_ЧАСТЬ_1_КЛАСТЕР/02_1.2_hello_ansible"
rm -rf /tmp/night_office
cat inventory.ini
ansible -i inventory.ini all -m ansible.builtin.ping
ansible-playbook -i inventory.ini playbook.yml
cat /tmp/night_office/prays.txt
```

✅ После команды

Ожидая: localhost отвечает `pong`, первый `PLAY RECAP` даёт `failed=0` и `changed=4`.

🗣 ГОВОРЮ

«Первый запуск изменил четыре вещи. Запускаю тот же playbook второй раз. Если состояние уже нужное, модули не делают лишнюю работу».

📄 V2-C1-S03-C02 · Второй play ничего не меняет

```
ansible-playbook -i inventory.ini playbook.yml
```

✅ После команды

Ожидая: `changed=0` и handler не запускается. Это и есть проверяемая идемпотентность.

🗣 ГОВОРЮ

«Теперь меняю одну входную переменную. Ansible обновляет только зависящий от неё файл и запускает handler. Это полезнее, чем всегда выполнять `systemctl restart`: сервис перезапускается только после реального изменения конфигурации».

📄 V2-C1-S03-C03 · Изменение переменной вызывает только нужные изменения

```
sed -i '' 's/: 4000/: 6000/' vars/prices.yml
ansible-playbook -i inventory.ini playbook.yml
cat /tmp/night_office/rassylka.txt
sed -i '' 's/: 6000/: 4000/' vars/prices.yml
rm -rf /tmp/night_office
```

✅ После команды

Ожидая: `changed=2`, handler срабатывает, итоговая сумма меняется; в конце исходное значение возвращено.

🧠 ЧТО ЭТО ЗНАЧИТ

Inventory отвечает «где», playbook — «какой сценарий», role — «какой переиспользуемый компонент», module — «какое состояние получить». Идемпотентность — проверяемый второй запуск с `changed=0`, а не обещание в README.

🏭 В ПРОДЕ

Inventory может быть динамическим, секреты идут из Vault/secret manager, а запуск — из CI или runner с ограниченным доступом. Если локально playbook работает, а в CI получает 403, сначала проверяют identity, route, allowlist и секреты; пустые логи приложения в такой ситуации ничего не говорят, потому что приложение могло вообще не стартовать.

ПЕРЕХОД

«Теперь открываем Kubespray. Это не отдельный бинарник-установщик, а большой Ansible-проект».

Время: 29:00–39:00

ЭКРАН: `kubespray` и pinned upstream `kubespray/vendor/kubespray`.

ГОВОРЮ

«Мы закрепили Kubespray на теге `v2.31.0`. В свой репозиторий не копируем весь чужой проект и не зависим от случайной ветки master: сохраняем версию, свой inventory, group_vars и загрузочный backstage-механизм.

Открываю верхний `cluster.yml`. Он импортирует playbooks. Дальше обычные роли: container engine, etcd, control plane, worker, network plugin. Наши решения лежат не в переписанном Kubespray, а в inventory и group_vars».

V2-C1-S04-C01 · Kubespray — обычный большой Ansible-проект

```
cd "$REPO_ROOT"
KUBESPRAY_ROOT="$REPO_ROOT/kubespray"
UPSTREAM="$KUBESPRAY_ROOT/vendor/kubespray"
cat "$KUBESPRAY_ROOT/VERSION"
sed -n '1,180p' "$UPSTREAM/cluster.yml"
find "$UPSTREAM/roles" -mindepth 1 -maxdepth 1 -type d -print | sort | sed -n
'1,24p'
sed -n '1,180p' "$KUBESPRAY_ROOT/inventory/video2/inventory.example.ini"
sed -n '1,180p' "$KUBESPRAY_ROOT/inventory/video2/group_vars/k8s_cluster/k8s-
cluster.yml"
```

После команды

Ожидая: pinned tag `v2.31.0`, верхнеуровневый `cluster.yml`, обычные роли, inventory и group_vars. Здесь ничего не устанавливаем.

ПОКАЗАТЬ

- `kubespray/VERSION`.
- `vendor/kubespray/cluster.yml`.
- Несколько ролей, не листать весь каталог.
- `inventory.example.ini`: группы нод.
- `group_vars/k8s_cluster/k8s-cluster.yml`: версия, runtime, CNI, CIDR, strictARP.

ЧТО ЭТО ЗНАЧИТ

Kubespray использует kubeadm под капотом, но добавляет проверенную структуру ролей и настройки для нескольких дистрибутивов. Это всё ещё Ansible: inventory → variables → roles → playbooks.

ЛОВУШКА

Не начинать установку здесь. Сначала нужны машины и проверенный inventory.

ПЕРЕХОД

«Посмотрели, чем машины будут настроены. Теперь разберёмся, откуда сами машины берутся».

Время: 39:00–55:00 плюс создание стенда

ЭКРАН: обычный урок BoostMentor → пять VM → форма создания VM в Yandex Cloud.

ГОВОРЮ

«Сначала беру рабочий стенд курса. Открываю обычный урок

<https://boostmentor.ru/dashboard/courses/devops-school/lesson/g8-project-cluster-lab1>, перехожу в практику и запускаю одно окружение. Оно создаёт пять чистых VM: node1–node3 для Kubespray и lb1–lb2 для внешнего HA. Перед стартом смотрю реальный таймер в UI и выбираю максимальный доступный срок. Называю в кадре только то, что фактически показано.

У задания внутри урока есть свой самостоятельный сценарий. В этом видео от платформы нужен только инфраструктурный контракт: пять чистых VM, SSH-адреса и ключ. Роли нод, containerd, версии, сети и Calico задаёт наш pinned inventory с group_vars — его я сейчас и показываю.

После нажатия не заполняю эфир ожиданием. Говорю “АНТРАКТ”, останавливаю REC, жду готовность пяти VM, скачиваю ключ и вне захвата заново ввожу пять SSH-адресов в `recording/.recording.env`. Возвращаюсь только когда SSH доступен. Если до конца таймера меньше восьми часов, запись Ч1 делю в два захода по чекпойнту перед C1.4-бис; не надеюсь, что среда сама проживёт дольше заявленного срока.

Теперь открываю облачную консоль. Одну VM можно создать руками: выбрать образ, CPU, память, диск, сеть, security group, SSH-ключ. Я пройду форму и не нажму кнопку создания. Для пяти машин ручной путь уже неудобен: легко получить разные диски, разные правила или забыть, почему одна нода отличается.

Terraform фиксирует те же решения в коде. UI полезен, чтобы увидеть реальные ресурсы и проверить результат. Но постоянные изменения должны возвращаться в код, иначе следующий plan покажет drift».

«Перед Terraform обновляю короткоживущий IAM-токен из активного профиля `yc`. Token, cloud ID и folder ID передаются провайдеру только через окружение текущего терминала: в код, state и Git они не попадают. Сначала один безопасный read-запрос к Compute Cloud — если он не проходит, apply не начинаю».

V2-C1-S05-C01 · До Terraform проверить облачный UI без второго ручного стенда

```
export YC_TOKEN="$(yc iam create-token)"
export YC_CLOUD_ID="$(yc config get cloud-id)"
export YC_FOLDER_ID="$(yc config get folder-id)"
test -n "$YC_TOKEN" && test -n "$YC_CLOUD_ID" && test -n "$YC_FOLDER_ID"
yc compute instance list --folder-id "$YC_FOLDER_ID" --format json \
  | jq -r '.[[] | [.name, .status] | @tsv'
printf 'Yandex CLI and Terraform environment: OK\n'
```

После команды

Ожидаю: свежий IAM-токен остаётся только в окружении текущего терминала, CLI отвечает, затем виден либо пустой список, либо существующие учебные VM. В браузере руками открываем Compute Cloud и только показываем поля создания VM — второй комплект ресурсов кликами не создаём.

ПОКАЗАТЬ В БРАУЗЕРЕ

- BoostMentor → курс DevOps → урок `g8-project-cluster-lab1` → «Открыть лабораторию».
- Одна кнопка запуска; после ожидания карточки node1–node3 и lb1–lb2.
- Yandex Compute Cloud → Create VM.
- Ubuntu 24.04, CPU/RAM/disk.
- Network interface, private/public IP, security group, SSH public key.

- Уйти без создания второй ручной VM.



В ПРОДЕ

Ручное создание иногда используют для диагностики или временного ресурса, но затем либо удаляют его, либо backport'ят изменение в Terraform. Иначе репозиторий перестаёт описывать среду.

ПЕРЕХОД

«Открываю self-managed Terraform и иду по структуре файлов, а не по одной портянке».

Время: 55:00–78:00 плюс ожидание apply/destroy

ЭКРАН: `infra/self-managed/terraform`.

ГОВОРЮ

«Теперь покажу код, который описывает эквивалентный пяти-VM стенд. Для дальнейших лабораторий остаются машины из урока; Terraform-стенд создаю временно, проверяю в UI и удаляю своим же state.

Здесь Terraform разложен как нормальный инфраструктурный проект.

`versions.tf` фиксирует Terraform и провайдер. `providers.tf` настраивает провайдер. `variables.tf` объявляет типизированные входы и запрещает открыть SSH всему интернету. `locals.tf` описывает ровно пять ролей: node1–node3 для Kubernetes, lb1–lb2 для внешнего входа. `network.tf` и `security_groups.tf` отвечают за сеть. `compute.tf` вызывает модуль. В `modules/k8s_nodes` находятся переиспользуемые ресурсы VM, дисков и интерфейсов. `outputs.tf` не просто печатает IP: он формирует два inventory — для Kubespray и для внешнего HA.

`terraform.tfvars.example` безопасно коммитить как образец. Реальные CIDR и SSH-ключ идут в локальный `terraform.tfvars`. State, приватные ключи и созданные inventory Git игнорирует».

ПОКАЗАТЬ

Идти в указанном порядке:

- `versions.tf`
- `providers.tf`
- `variables.tf`

- `locals.tf`
- `network.tf` и `security_groups.tf`
- `compute.tf`
- `modules/k8s_nodes/main.tf`
- `outputs.tf`
- `backend.s3.tf.example`

📖 V2-C1-S06-C01 · Проверить production-shaped Terraform до apply

```
cd "$REPO_ROOT/infra/self-managed/terraform"
test -f terraform.tfvars || cp terraform.tfvars.example terraform.tfvars
terraform fmt -check -recursive
terraform init
terraform validate
terraform plan -out=/tmp/video2-self-managed.tfplan >/tmp/video2-self-managed-
plan.log
terraform show -json /tmp/video2-self-managed.tfplan \
  | jq -r '.resource_changes[] | [.address, (.change.actions | join(", "))] |
@tsv'
```

✅ После команды

Ожидаю: план ровно пяти VM — `node1`, `node2`, `node3`, `lb1`, `lb2` — плюс сеть, подсеть и правила доступа. В `terraform.tfvars` уже должны быть твой public IP `/32` и SSH public key.

🗣 ГОВОРЮ

«Plan сохраняю и вывожу безопасной таблицей `address → action`: так вижу пять VM и зависимые ресурсы без folder ID и адресов. Apply использует именно этот сохранённый plan. После apply Terraform отдаёт публичные адреса для SSH, private

addresses для межмашинного трафика и умеет собрать inventory, но в кадре я заменяю адреса и instance ID на `<redacted>`. Так выглядит связка Terraform → Ansible в инфраструктуре, которую контролирует наша команда.

После apply переключаюсь в Yandex Cloud и проверяю, что код действительно создал: сеть, подсеть, security group и пять VM. В UI ничего не исправляю».

📄 V2-C1-S06-C02 · Временно создать пять VM и проверить Terraform outputs

```
terraform apply -no-color /tmp/video2-self-managed.tfplan \  
  >/tmp/video2-self-managed-apply.log  
grep -E '^(Apply complete|Outputs:|[[[:alnum:]]_]+ = <sensitive>)' \  
  /tmp/video2-self-managed-apply.log  
terraform output -json nodes | jq 'with_entries(.value |= {  
  roles,  
  external_ip: "<redacted>",  
  internal_ip: "<redacted>",  
  instance_id: "<redacted>"  
})'  
terraform output -raw kubespray_inventory \  
  | sed -E 's/(ansible_host|ip|access_ip)=[^ ]+/\1=<redacted>/g'  
terraform output -raw external_ha_inventory \  
  | sed -E 's/(ansible_host|private_ip)=[^ ]+/\1=<redacted>/g'  
yc compute instance list --folder-id "$YC_FOLDER_ID" --format json \  
  | jq -r '.[ ] | [.name, .status] | @tsv'
```

✅ После команды

Ожидаю: временные `node1-node3` и `lb1-lb2`, сеть и два inventory как outputs. В кадре видны роли и структура, а IP/instance ID заменены на `<redacted>`. Рабочим стендом записи остаются пять VM из урока BoostMentor.

✅ БРАУЗЕРНАЯ ПРОВЕРКА

- Compute Cloud → пять VM Running.
- VPC → нужна сеть/подсеть.
- У каждой VM private IP; public IP используется для SSH.
- Не показывать project/folder IDs крупным планом.

🗣️ ГОВОРЮ

«Временный стенд задачу выполнил. Сначала читаю destroy plan, затем удаляю только ресурсы этого state. Учебные VM из BoostMentor — другой жизненный цикл, они продолжают работать».

📄 V2-C1-S06-C03 · Удалить только временный Terraform-стенд

```
terraform plan -destroy -out=/tmp/video2-self-managed-destroy.tfplan \  
  >/tmp/video2-self-managed-destroy-plan.log  
terraform show -json /tmp/video2-self-managed-destroy.tfplan \  
  | jq -r '.resource_changes[] | [.address, (.change.actions | join(","))] |  
@tsv'  
terraform apply -no-color /tmp/video2-self-managed-destroy.tfplan \  
  >/tmp/video2-self-managed-destroy.log  
grep -E '^Destroy complete' /tmp/video2-self-managed-destroy.log  
yc compute instance list --folder-id "$YC_FOLDER_ID" --format json \  
  | jq -r '.[ ] | [.name, .status] | @tsv'
```

✅ После команды

Ожидаю: Terraform удаляет только ресурсы этого state. Пять VM учебной платформы живут отдельно и остаются для Kubespray и внешнего HA.

⚠️ ЕСЛИ APPLY ДОЛГО ИДЁТ

Сказать «АНТРАКТ», остановить REC, дожждаться завершения. После возврата показать последние строки apply и outputs. Не заполнять ожидание импровизацией.

ПЕРЕХОД

«Self-managed машины уже дала учебная лаборатория, а Terraform показал их production-shaped описание. Рядом создадим managed: Terraform похож, но граница ответственности другая».

Время: 78:00–93:00 плюс ожидание apply

ЭКРАН: `infra/managed/terraform` и Yandex Cloud.

ГОВОРЮ

«Managed-кластер — отдельный Terraform root и отдельный state. Здесь есть сеть, service accounts, кластер и node group. Control plane создаёт и обслуживает провайдер, workers остаются видимыми ресурсами.

Открываю файлы по порядку: `main.tf` с версиями, `network.tf`, `service-accounts.tf`, `kubernetes.tf` и `outputs.tf`. Сетевые диапазоны у managed отдельные, чтобы стенды не пересекались.

В `private.auto.tfvars` лежат локальные CIDR и разрешённый адрес `/32`. Этот файл не коммитим. Plan сверяю до apply».

ПОКАЗАТЬ

- `main.tf`: provider/version/variables.
- `network.tf`: VPC, subnet, security group.
- `service-accounts.tf`.
- `kubernetes.tf`: cluster и node group.
- `outputs.tf`: CIDR и команда получения credentials.

V2-C1-S07-C01 · Managed-кластер тем же IaC-подходом

```
cd "$REPO_ROOT/infra/managed/terraform"
test -f private.auto.tfvars || cp private.auto.tfvars.example private.auto.tfvars
terraform fmt -check -recursive
terraform init
terraform validate
terraform plan -out=/tmp/video2-managed.tfplan >/tmp/video2-managed-plan.log
terraform show -json /tmp/video2-managed.tfplan \
  | jq -r '.resource_changes[] | [.address, (.change.actions | join(","))] | @tsv'
terraform apply -no-color /tmp/video2-managed.tfplan >/tmp/video2-managed-apply.log
grep -E '^(Apply complete|Outputs:|[:alnum:])+ = <sensitive>' \
  /tmp/video2-managed-apply.log
yc managed-kubernetes cluster list --format json \
  | jq -r '.[0] | [.name, .status] | @tsv'
yc managed-kubernetes node-group list --format json \
  | jq -r '.[0] | [.name, .status] | @tsv'
terraform output cidrs
```

После команды

Ожидая: managed control plane и node group. В браузере проверяем те же ресурсы в Yandex Cloud, не редактируем их руками.

БРАУЗЕРНАЯ ПРОВЕРКА

- Managed Service for Kubernetes → Cluster status Running.
- Node group status Ready.
- VPC/subnet с ожидаемым CIDR.
- Service accounts.

ЧТО ЭТО ЗНАЧИТ

Terraform используется и для managed, и для self-managed. Отличается объект управления: в одном случае Terraform создаёт обычные VM, а Kubernetes устанавливаем мы; в другом он вызывает API провайдера, который управляет control plane.

ПЕРЕХОД

«Managed пока достраивается. Возвращаюсь к трём обычным Linux-машинам и проверяю их до Kubespray».

Время: 93:00–108:00

ЭКРАН: `kubespray/inventory/video2/inventory.example.ini`, безопасный `ansible-inventory --graph` и терминал.

ГОВОРЮ

«Пять VM уже созданы одной лабораторией. После запуска я вне кадра вписал их SSH-адреса и путь к скачанному ключу в локальный env. Теперь прямым here-doc собираю два разных inventory: три Kubernetes-ноды для Kubespray и все пять машин для внешнего HA. Никакого скрытого генератора здесь нет.

В Kubernetes inventory node1 и node2 входят в `kube_control_plane`. Все три — в `etcd`. node2 и node3 — workers. node1 workload не принимает: так на маленьком стенде у нас два API-сервера, три члена etcd и два места для приложения.

`ansible_host` — адрес, по которому мой ноутбук подключается по SSH. `ip` и `access_ip` здесь не задаю: сначала Ansible снимает реальный private IPv4 из facts, и этот адрес используется для трафика между VM. Floating IP может вообще не быть назначен интерфейсу внутри VM.

На экране открываю только committed example и graph без hostvars. Реальный inventory нужен Ansible для подключения, но пять floating-адресов в запись не вывожу.

Для локального `admin.conf` включён `kubeconfig_localhost_ansible_host`: endpoint в артефакте будет указывать на `ansible_host` node1. Public SAN нужен, чтобы сертификат этот адрес принимал. Это лабораторный single-endpoint доступ, не HA endpoint для production.

Затем проверяю не только ping, но и sudo. Если доступ или become не работают, бессмысленно запускать двухчасовой playbook».

V2-C1-S08-C01 · Прочитать роли нод и проверить SSH/sudo

```
cd "$REPO_ROOT"
KUBESPRAY_ROOT="$REPO_ROOT/kubespray"
INV="$KUBESPRAY_ROOT/inventory/video2/inventory.ini"
source "$REPO_ROOT/recording/.recording.env"
SSH_KEY="$VIDEO2_SSH_KEY"
SSH_USER="${VIDEO2_SSH_USER:-root}"
: "${NODE1_SSH_HOST:?fill recording/.recording.env}"
: "${NODE2_SSH_HOST:?fill recording/.recording.env}"
: "${NODE3_SSH_HOST:?fill recording/.recording.env}"
: "${LB1_SSH_HOST:?fill recording/.recording.env}"
: "${LB2_SSH_HOST:?fill recording/.recording.env}"
install -d -m 0700 "$KUBESPRAY_ROOT/inventory/video2/group_vars/all"
cat > "$INV" <<EOF
[all]
node1 ansible_host=$NODE1_SSH_HOST ansible_user=$SSH_USER
node2 ansible_host=$NODE2_SSH_HOST ansible_user=$SSH_USER
node3 ansible_host=$NODE3_SSH_HOST ansible_user=$SSH_USER

[kube_control_plane]
node1
node2

[etcd]
node1
node2
node3

[kube_node]
node2
node3

[k8s_cluster:children]
```

```
kube_control_plane
kube_node
EOF
cat > "$KUBESPRAY_ROOT/inventory/video2/group_vars/all/all.yml" <<EOF
---
supplementary_addresses_in_ssl_keys:
  - $NODE1_SSH_HOST
EOF
cat > "$REP0_ROOT/ansible/external-ha/inventory.ini" <<EOF
[load_balancers]
lb1 ansible_host=$LB1_SSH_HOST ansible_user=$SSH_USER ha_overlay_ip=10.77.0.11
ha_priority=150
lb2 ansible_host=$LB2_SSH_HOST ansible_user=$SSH_USER ha_overlay_ip=10.77.0.12
ha_priority=100

[kubernetes_nodes]
node1 ansible_host=$NODE1_SSH_HOST ansible_user=$SSH_USER
node2 ansible_host=$NODE2_SSH_HOST ansible_user=$SSH_USER
node3 ansible_host=$NODE3_SSH_HOST ansible_user=$SSH_USER

[all:vars]
ansible_python_interpreter=/usr/bin/python3
ha_vip=$HA_VIP
ha_vip_prefix=24
ha_frontend_port=$HA_PORT
k8s_nodeport=$K8S_NODEPORT
EOF
chmod 600 "$SSH_KEY"
source "$KUBESPRAY_ROOT/.venv/bin/activate"
sed -n '1,160p' "$KUBESPRAY_ROOT/inventory/video2/inventory.example.ini"
ansible-inventory -i "$INV" --graph
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.ping
```

```
ansible -i "$INV" all --private-key "$SSH_KEY" --become -m  
ansible.builtin.command -a whoami
```

✓ После команды

Ожидаю: структура видна по безопасному example и graph без hostvars; два реальных inventory построены из пяти адресов одной учебной лаборатории, но адреса не печатаются. node1/node2 в control plane, все три в etcd, node2/node3 в workers; все отвечают `pong` и `root` через become.

ГОВОРЮ

«Теперь три причины типичного падения установки: включённый swap, неправильный cgroup и отсутствующий маршрут. Последней командой Ansible сам собирает private IPv4 facts. Локальный успех здесь ещё не гарантирует успех в CI: runner может идти с другого адреса, не иметь route или нужного ключа. Поэтому фиксируем, откуда идёт запуск».

☰ V2-C1-S09-C01 · Проверить Linux-предпосылки и private networking

```
ansible -i "$INV" all --private-key "$SSH_KEY" --become -m ansible.builtin.shell \
  -a 'test -z "$(swapon --show --noheadings)"; test "$(stat -fc %T  
/sys/fs/cgroup)" = cgroup2fs; ip -4 route show default'
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.setup \
  -a 'filter=ansible_default_ipv4'
```

✓ После команды

Ожидаю: swap пуст, cgroup v2, default route есть; `ansible_default_ipv4.address` — private IP. Public/floating IP применяется только для SSH.

⚠ ЕСЛИ PING НЕ ПРОШЁЛ

Проверить по порядку: VM Running → public IP → security group /32 → имя пользователя → ключ → route/VPN. Не отключать VPN и не менять системную сеть. Пустые логи Kubernetes здесь нормальны: Kubernetes ещё не установлен.

ПЕРЕХОД

«Доступ есть, роли назначены, Linux готов. Теперь открываю параметры, которые Kubespray применит, и только после этого запускаю установку».

Время: 108:00–131:00 плюс 30–60 минут ожидания

ЭКРАН: upstream Kubespray, наши group_vars, терминал.

ГОВОРЮ

«Перед командой читаю два слоя.

В upstream `cluster.yml` — вход в проект и импорты playbooks. В нашем `k8s-cluster.yml` — решения конкретного кластера: Kubernetes `v1.34.7`, containerd, Calico, Pod CIDR, Service CIDR и `kube_proxy_strict_arp: true` для MetalLB.

Inventory отвечает, где живёт компонент. Group vars отвечают, с какими параметрами его ставить. Roles выполняют работу».

V2-C1-S09-C02 · Перед запуском прочитать playbook и параметры кластера

```
UPSTREAM="$KUBESPRAY_ROOT/vendor/kubespray"
sed -n '1,220p' "$UPSTREAM/cluster.yml"
sed -n '1,220p' "$UPSTREAM/playbooks/cluster.yml"
sed -n '1,220p' "$KUBESPRAY_ROOT/inventory/video2/group_vars/k8s_cluster/k8s-cluster.yml"
find "$UPSTREAM/roles" -mindepth 1 -maxdepth 1 -type d -print | sort | sed -n '1,24p'
```

После команды

Ожидаю: видны `kube_version`, containerd, Calico, Pod/Service CIDR, strictARP, `kubeconfig_localhost` и `kubeconfig_localhost_ansible_host`. Последняя настройка заставит локальный `admin.conf` использовать SSH-адрес node1, а не недоступный с ноутбука private IP. Никакой установки ещё не было.

ГОВОРЮ

«Главный запуск — обычный `ansible-playbook` на upstream `cluster.yml`. Не прячу его в `deploy.sh`. Мы видим задачи runtime, etcd, control plane, kubelet и CNI.

Процесс долгий. Я не буду читать полчаса бегущие строки. Отмечаю начало, говорю «АНТРАКТ», выключаю запись и возвращаюсь на `PLAY RECAP`. Если есть `failed`, это отдельный troubleshooting-дубль, а не повод делать вид, что всё готово.

После успешного ресап Kubespray оставляет admin kubeconfig с настоящим CA. Я добавляю его в общий config под понятным контекстом `kubespray`. TLS-проверку не отключаю».

V2-C1-S09-C03 · Прямой запуск Kubespray и безопасный kubeconfig

```
PLAYBOOK="$KUBESPRAY_ROOT/.venv/bin/ansible-playbook"
pushd "$UPSTREAM" >/dev/null
"$PLAYBOOK" -i "$INV" cluster.yml --become --private-key "$SSH_KEY"
popd >/dev/null
KUBECONFIG_FILE="$KUBESPRAY_ROOT/inventory/video2/artifacts/admin.conf"
OLD_CONTEXT="$(KUBECONFIG="$KUBECONFIG_FILE" kubectl config current-context)"
KUBECONFIG="$KUBECONFIG_FILE" kubectl config rename-context "$OLD_CONTEXT"
kubespray
mkdir -p "$HOME/.kube"
KUBECONFIG="$HOME/.kube/config:$KUBECONFIG_FILE" kubectl config view --flatten >
/tmp/video2-kubeconfig
install -m 0600 /tmp/video2-kubeconfig "$HOME/.kube/config"
kubectl --context kubespray get nodes -o wide
```

После команды

Ожидая: `PLAY RECAP` без failed, затем три Ready-ноды. TLS не отключаем и `--insecure-skip-tls-verify` не используем.

✅ ПОКАЗАТЬ ПОСЛЕ ОЖИДАНИЯ

- Последние задачи и `PLAY RECAP`.
- `failed=0`.
- Три Ready-ноды и их версии/roles.
- Не показывать содержимое client-key-data.

🧠 ЧТО ФИЗИЧЕСКИ ПОЯВИЛОСЬ

На node1/node2 — API server и control-plane компоненты; на всех трёх — etcd; на node2/node3 — kubelet для workload; на каждой нужной ноде — containerd и Calico. До этой команды были просто Linux-VM.

ПЕРЕХОД

«Ready в `get nodes` — необходимая, но слабая проверка. Теперь докажем API, системные компоненты, сеть и реальный HTTP».

Время: 131:00–142:00

ЭКРАН: терминал → `kubernetes/devops-may-cry/base`.

ГОВОРЮ

«Проверка идёт слоями. Сначала ноды. Затем endpoint `/readyz` самого API server. Потом системные Pod'ы, DaemonSet Calico и Deployment CoreDNS.

После инфраструктуры ставлю наше приложение из base Kustomize. В манифесте нет кода Python внутри YAML. Это отдельный immutable multi-arch образ Go-приложения, security context, probes, requests/limits, PDB и spread по hostname».

ПОКАЗАТЬ

- `kubernetes/devops-may-cry/base/deployment.yaml`.
- image digest, не плавающий latest.
- startup/liveness/readiness probes.
- `runAsNonRoot`, read-only filesystem, drop capabilities.
- topology spread и PDB.

V2-C1-S10-C01 · Доказать, что кластер действительно работает

```
kubectl --context kubespray get nodes -o wide
kubectl --context kubespray get --raw='/readyz?verbose' | tail -12
kubectl --context kubespray -n kube-system get pods -o wide
kubectl --context kubespray -n kube-system get daemonset calico-node
kubectl --context kubespray -n kube-system get deployment coredns
kubectl --context kubespray apply -k "$REPO_ROOT/kubernetes/devops-may-cry/base"
kubectl --context kubespray -n traffic-lab rollout status deployment/devops-may-cry --timeout=180s
kubectl --context kubespray -n traffic-lab get pods,service,endpointslice -o wide
```

✅ После команды

Ожидаю: API ready, системные Pod'ы Running, Calico на каждой ноде, CoreDNS доступен, приложение разложено по workers и EndpointSlice содержит ready endpoints.

🗣 ГОВОРЮ

«ClusterIP с ноутбука напрямую недоступен, поэтому создаю одноразовый curl-клиент внутри кластера. Он резолвит имя Service и вызывает readiness endpoint. После доказательства удаляю клиент».

📄 V2-C1-S10-C02 · Проверить Service из временного клиента

```
kubectl --context kubespray -n traffic-lab run verify-client \
  --image=curlimages/curl:8.12.1 --restart=Never --command -- sleep 3600
kubectl --context kubespray -n traffic-lab wait --for=condition=Ready pod/verify-
client --timeout=120s
kubectl --context kubespray -n traffic-lab exec verify-client -- curl -fsS
http://devops-may-cry/readyz
kubectl --context kubespray -n traffic-lab delete pod verify-client --wait=true
```

✅ После команды

Ожидаю: HTTP JSON со `status=ok`, затем одноразовый клиент удалён.

🧠 ЧТО ЭТО ДОКАЗЫВАЕТ

`get nodes` доказывает регистрацию kubelet. `/readyz` — состояние API server. System pods — базовые addons. Rollout + EndpointSlice + HTTP 200 — что scheduler, runtime, CNI, DNS и Service вместе довели запрос до приложения.

🚧 В ПРОДЕ

Если Pod не стартовал и application logs пусты, сначала смотрят Events, image pull, admission и scheduling. Ошибка registry auth или недоступный external service происходит до запуска процесса, поэтому пустой stdout не оправдывает гипотезу «сломался код приложения».

ПЕРЕХОД

«Кластер работает. Теперь на три минуты отступим назад и разберём, откуда вообще берутся маски и диапазоны. Без этого Pod CIDR выглядит как набор случайных цифр».

Время: 142:00–154:00

ЭКРАН: доска «Маска подсети: как считать»; при наличии — «От сигнала к биту».

ГОВОРЮ

«Начнём с самого низа. Транзистор можно использовать как быстрый переключатель: условно есть сигнал — один, нет сигнала — ноль. Один бит даёт два состояния. Два бита — четыре: 00, 01, 10, 11. Каждый следующий бит удваивает число комбинаций. Поэтому для n бит получаем два в степени n .

IPv4-адрес — 32 бита, которые для удобства записали четырьмя десятичными октетами. В каждом октете 8 бит, поэтому значений 256 — от нуля до 255.

Префикс говорит, сколько первых бит относится к сети. Для `/26` сетевых бит 26, свободных — 6. Два в шестой степени — 64 адреса. В сети `192.168.1.70/26` адрес .70 лежит в блоке .64–.127. .64 — адрес сети, .127 — broadcast, обычные хосты — .65–.126.

В Kubernetes отдельно выбираем subnet под, Pod CIDR и Service CIDR. Они не должны пересекаться друг с другом, VPC, офисной сетью и VPN. Потом менять их тяжело, поэтому это решение принимают до установки».

V2-C1-S11-C01 · Посчитать сеть на конкретном IPv4-примере

```
python3 - <<'PY'
import ipaddress

for value in ("192.168.1.70/26", "10.233.64.0/18", "10.233.0.0/18"):
    net = ipaddress.ip_network(value, strict=False)
    print(value)
    print("  network    ", net.network_address)
    print("  netmask    ", net.netmask)
    print("  broadcast  ", net.broadcast_address)
    print("  addresses  ", net.num_addresses)
    print("  first/last", net.network_address + 1, net.broadcast_address - 1)
PY
```

✅ После команды

Ожидая для `192.168.1.70/26`: сеть `192.168.1.64`, usable `.65–.126`, broadcast `.127`, всего 64 адреса.

🧠 ЧТО ЭТО ЗНАЧИТ

Маска — не случайный набор чисел вроде `255`. Это двоичная граница между сетевой и хостовой частью. `2^(32-prefix)` даёт размер блока; usable host range для обычной IPv4 subnet часто исключает network и broadcast, но Kubernetes и облака могут иметь свои правила резервирования.

⚠️ НЕ УПРОЩАТЬ ДО ОШИБКИ

Не говорить, что любой `/24` всегда даёт ровно 254 доступных адреса приложению: облачный провайдер может резервировать дополнительные IP, а Pod networking работает по своей модели.

ПЕРЕХОД

«Теперь посмотрим, как это проявилось в только что созданном кластере».

Время: 154:00–171:00

ЭКРАН: сначала доска «Calico vs Cilium», затем «Как работает Calico», затем терминал.

ГОВОРЮ

«Calico и Cilium решают базовую задачу CNI, но делают это разными dataplane и наборами возможностей. На нашей доске сравнение упрощённое: Calico здесь — привычный выбор с NetworkPolicy и маршрутизацией, Cilium — eBPF-native, Hubble и более глубокая L7-наблюдаемость. Cilium может заменить kube-proxy в соответствующей конфигурации, но это не автоматическое свойство любого кластера.

Наш стенд установлен с Calico. У CNI три базовых действия: подключить Pod к сети, назначить IP и обеспечить маршрут до других Pod'ов.

Внутри Pod виден `eth0`. На ноде ему соответствует veth-интерфейс. Calico IPAM выдаёт адреса из pool блоками, закреплёнными за нодами. Между нодами трафик идёт по маршруту и, в зависимости от конфигурации, напрямую, через IP-in-IP или VXLAN.

Сначала отделяю `Node.spec.podCIDR` от Calico IPAM block. На доске может быть пример блока `/26`, а на нашем кластере kube-controller-manager выдаёт нодам другие PodCIDR. Принцип тот же, но размер и сущность разные».

V2-C1-S12-C01 · Связать Node PodCIDR с реальными Pod IP

```
REPO_ROOT="$(git rev-parse --show-toplevel)"
LAB_DIR="$REPO_ROOT/01_ЧАСТЬ_1_КЛАСТЕР/09_1.7_cidr_to_node"
INV="$REPO_ROOT/kubespray/inventory/video2/inventory.ini"
SSH_KEY="${VIDEO2_SSH_KEY:-$HOME/.ssh/id_ed25519}"
kubectl --context kubespray get nodes \
  -o custom-columns='NODE:.metadata.name,INTERNAL-IP:.status.addresses[?
(@.type=="InternalIP")].address,POD-CIDR:.spec.podCIDR'
kubectl --context kubespray get ippools.crd.projectcalico.org -o yaml | \
  grep -E 'cidr:|blockSize:|ipipMode:|vxlanMode:'
kubectl --context kubespray get blockaffinities.crd.projectcalico.org \
  -o custom-columns='NODE:.spec.node,CIDR:.spec.cidr,STATE:.spec.state'
```

После команды

Ожидаю: три разных `Node.spec.podCIDR` и отдельно Calico pool/blockSize. Это связанные, но не одинаковые сущности.

ГОВОРЮ

«Теперь ставлю два диагностических Pod'а на разные worker-ноды. Показываю IP, `eth0` внутри первого Pod, ping до второго и хостовый маршрут. Если `vxlan.calico` отсутствует, я не объявляю VXLAN сломанным: сначала смотрю фактический режим IPPool. Команда специально допускает другой dataplane».

V2-C1-S12-C02 · Посмотреть veth, маршрут и overlay на живых Pod'ах

```
kubectl --context kubespray apply -f "$LAB_DIR/calico-proof.yaml"
kubectl --context kubespray -n network-proof wait --for=condition=Ready pod --all
--timeout=120s
kubectl --context kubespray -n network-proof get pods -o wide
POD_B_IP="$(kubectl --context kubespray -n network-proof get pod calico-b -o
jsonpath='{.status.podIP}')"
kubectl --context kubespray -n network-proof exec calico-a -- ip address show
eth0
kubectl --context kubespray -n network-proof exec calico-a -- ping -c 3
"$POD_B_IP"
source "$REPO_ROOT/kubespray/.venv/bin/activate"
ansible -i "$INV" node2 --private-key "$SSH_KEY" --become -m
ansible.builtin.shell \
    -a "ip route get $POD_B_IP; ip -o link | grep cali | head; ip -d link show
vxlan.calico || true"
kubectl --context kubespray delete namespace network-proof --wait=true
```

После команды

Ожидаю: Pod IP, `eth0` внутри Pod, `cali...`-интерфейсы/route на хосте и успешный ping между нодами. VXLAN показываем только если этот dataplane реально активен.

ЧТО ЭТО ДОКАЗЫВАЕТ

Связка «Pod IP → нода → veth → route → overlay/underlay» показана фактами живого кластера. Сравнительный слайд не заменяет эту проверку.

В ПРОДЕ

Менять CNI на работающем кластере дорого и рискованно. Обычно выбирают dataplane, MTU, encapsulation и policy model до создания workload. Для troubleshooting нужны не только `kubectl get pods`, но и IPPool, routes, interfaces, packet capture и события.

ПЕРЕХОД

«Self-managed готов. Забираю доступ к managed и ставлю два кластера рядом».

Время: 171:00–177:00

ЭКРАН: терминал и доска «Managed vs Self-managed».

ГОВОРЮ

«Managed credentials выдаёт CLI провайдера. Self-managed kubernetes создал Kubespray с нашим CA и public SAN. Я называю контексты `yc-managed` и `kubespray`, чтобы команда всегда явно показывала целевой кластер.

Не использую `--insecure-skip-tls-verify`. Если сертификат не подходит адресу API, исправляют SAN/endpoint или способ доступа, а не выключают проверку».

V2-C1-S13-C01 · Получить managed kubernetes и назвать контексты

```
cd "$REPO_ROOT/infra/managed/terraform"
$(terraform output -raw get_credentials_command)
kubectl config rename-context "$(kubectl config current-context)" yc-managed
kubectl --context yc-managed get nodes -o wide
kubectl config get-contexts
```

После команды

Ожидая: `kubespray` и `yc-managed` существуют одновременно, а у managed видны только worker-ноды.

СЕКРЕТЫ В КАДРЕ

Не раскрывать `client-key-data`, токены, содержимое реального kubernetes и cloud IDs. Для объяснения достаточно имени контекста, cluster server можно замаскировать.

ПЕРЕХОД

«Перед сравнением сделаем то, что в эксплуатации self-managed неизбежно: обслужим ноду и реально сменим версию».

Время: 177:00–188:00

ЭКРАН: доска «Безопасное обновление» → терминал.

ГОВОРЮ

«Сначала отдельно показываю механику обслуживания. `cordon` запрещает новые Pod'ы на ноде. `drain` выселяет управляемые workload, уважает PDB и пропускает DaemonSet. `uncordon` возвращает ноду scheduler'у.

Это ещё не upgrade. Версия kubelet и control plane от этих команд не меняется».

V2-C1-S14-C01 · Maintenance drill: cordon, drain, PDB, uncordon

```
kubectl --context kubespray apply -k "$REPO_ROOT/kubernetes/devops-may-cry/base"
kubectl --context kubespray -n traffic-lab rollout status deployment/devops-may-cry --timeout=180s
kubectl --context kubespray cordon node2
kubectl --context kubespray -n traffic-lab get pods -l app=devops-may-cry -o wide
kubectl --context kubespray drain node2 --ignore-daemonsets --delete-emptydir-data
kubectl --context kubespray -n traffic-lab get pods -l app=devops-may-cry -o wide
kubectl --context kubespray -n traffic-lab get pdb devops-may-cry
kubectl --context kubespray uncordon node2
```

После команды

Ожидая: node2 становится unschedulable, обычные Pod'ы переезжают с учётом PDB, DaemonSet не выселяется, затем node2 возвращается.

ЧТО ПРОВЕРИТЬ

Pod'ы приложения должны остаться доступны. Если PDB блокирует drain, не добавлять `--disable-eviction` ради красивого дубля: сначала объяснить, какое availability-ограничение мешает.

ПЕРЕХОД

«Теперь готовим именно смену версии: состояние до, snapshot etcd, diff конфигурации, непрерывная проба, upgrade playbook и доказательство после».

C1.8-A Настоящий upgrade 1.34.7 → 1.35.4

Время: 188:00–214:00 плюс ожидание

ЭКРАН: терминал, `kubespray/versions`, upstream `upgrade-cluster.yml`.

ГОВОРЮ

«Перед изменением снимаю факты: версии нод, готовность API и snapshot etcd. Snapshot не заменяет проверенный restore runbook, но без него начинать изменение control plane ещё хуже. Отдельно смотрю срок сертификатов».

V2-C1-S15-C01 · До upgrade: версии, API и snapshot etcd

```
KUBESPRAY_ROOT="$REPO_ROOT/kubespray"
UPSTREAM="$KUBESPRAY_ROOT/vendor/kubespray"
INV="$KUBESPRAY_ROOT/inventory/video2/inventory.ini"
GROUP_VARS="$KUBESPRAY_ROOT/inventory/video2/group_vars/k8s_cluster/k8s-
cluster.yml"
SSH_KEY="${VIDE02_SSH_KEY:-$HOME/.ssh/id_ed25519}"
source "$KUBESPRAY_ROOT/.venv/bin/activate"
kubectl --context kubespray get nodes \
  -o custom-
columns='NODE:.metadata.name,READY:.status.conditions[-1].status,VERSION:.status.
nodeInfo.kubeletVersion'
kubectl --context kubespray get --raw='/readyz?verbose' | tail -12
ansible -i "$INV" node1 --private-key "$SSH_KEY" --become -m
ansible.builtin.shell -a \
  'install -d -m 0700 /var/backups/etcd; /usr/local/bin/etcdctl.sh --
endpoints=https://127.0.0.1:2379 snapshot save /var/backups/etcd/video2-pre-
1.35.4.db; /usr/local/bin/etcdutl snapshot status /var/backups/etcd/video2-pre-
1.35.4.db -w table'
ansible -i "$INV" node1 --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a '/usr/local/bin/kubeadm certs check-expiration'
```

✅ После команды

Ожидаю: baseline `v1.34.7`, API ready, snapshot со status table и срок сертификатов.

🗣 ГОВОРЮ

«Целевая версия заранее проверена в матрице именно этого Kubespray tag. Меняю одну строку через подготовленный version file и показываю diff. Не прыгаю через несколько minor-версий и не выдаю произвольный номер за поддерживаемый».

📄 V2-C1-S15-C02 · Показать единственную смену версии

```
cp "$KUBESPRAY_R00T/versions/k8s-cluster-1.35.4.yml" "$GROUP_VARS"  
git -C "$REPO_R00T" diff --  
kubespray/inventory/video2/group_vars/k8s_cluster/k8s-cluster.yml
```

✅ После команды

Ожидаю: содержательный diff `kube_version: 1.34.7 → 1.35.4`, а не россыпь несвязанных правок.

🗣 ГОВОРЮ

«Во втором окне оставляю port-forward, в третьем — curl раз в секунду. Это не обещание нулевого простоя. После прогона мы увидим наблюдаемое поведение и сможем отличить короткий разрыв от полной потери сервиса».

📄 V2-C1-S15-C03 · Непрерывная HTTP-проба во время upgrade

```
# Терминал 1:
kubectl --context kubespray -n traffic-lab port-forward service/devops-may-cry
18080:80 &
printf '%s\n' "$!" > /tmp/video2-upgrade-port-forward.pid
wait "$(cat /tmp/video2-upgrade-port-forward.pid)"
# Терминал 2:
while true; do date +%T; curl -fsS http://127.0.0.1:18080/readyz || echo FAIL;
sleep 1; done &
printf '%s\n' "$!" > /tmp/video2-upgrade-probe.pid
wait "$(cat /tmp/video2-upgrade-probe.pid)"
```

✅ После команды

Ожидаю: видимый поток `status=ok`. Это наблюдение, не обещание нулевого downtime.

🗣️ ГОВОРЮ

«Теперь запускаю upstream `upgrade-cluster.yml` напрямую. Kubespray обновляет компоненты по группам и нодам. Долгий вывод режу так же: начало → «АНТРАКТ» → возвращение на ресар».

📄 V2-C1-S15-C04 · Запустить настоящий upgrade playbook

```
PLAYBOOK="$KUBESPRAY_ROOT/.venv/bin/ansible-playbook"
pushd "$UPSTREAM" >/dev/null
"$PLAYBOOK" -i "$INV" upgrade-cluster.yml \
  --become --private-key "$SSH_KEY"
popd >/dev/null
```

✅ После команды

Ожидаю: последовательное обслуживание нод и `PLAY RECAP` без failed. Это уже upgrade, в отличие от одного `drain`.

ГОВОРЮ

«После resar не заканчиваю на слове changed. Снова проверяю версию каждой ноды, API ready, системные Pod'ы, rollout приложения и Warning events. Критерий завершения — сервис восстановлен и новая версия подтверждена. После проверки команда останавливает обе фоновые пробы и освобождает локальный порт».

V2-C1-S15-C05 · Доказать восстановление после upgrade

```
kubectl --context kubespray get nodes \
  -o custom-
columns='NODE:.metadata.name,READY:.status.conditions[-1].status,VERSION:.status.
nodeInfo.kubeletVersion'
kubectl --context kubespray get --raw='/readyz?verbose' | tail -12
kubectl --context kubespray -n kube-system get pods -o wide
kubectl --context kubespray -n traffic-lab rollout status deployment/devops-may-
cry --timeout=180s
kubectl --context kubespray -n traffic-lab get pods -o wide
kubectl --context kubespray get events -A --field-selector type=Warning --sort-
by=.lastTimestamp | tail -12
for pid_file in /tmp/video2-upgrade-probe.pid /tmp/video2-upgrade-port-
forward.pid; do
  if test -f "$pid_file"; then
    kill "$(cat "$pid_file")" 2>/dev/null || true
    rm -f "$pid_file"
  fi
done
```

После команды

Ожидаю: все ноды Ready на `v1.35.4`, API ready, workload доступен. Warning events читаем, а не скрываем. В конце обе upgrade-пробы остановлены, порт `18080` освобождён.

В ПРОДЕ

Перед upgrade дополнительно нужны release notes, deprecated APIs, backup/restore rehearsal, окно изменений, наблюдаемость и rollback/stop criteria. Safe workaround вроде временного preload image может спасти лабораторию, но не становится production-решением: долг фиксируют и закрывают нормальным registry/IAM.

ПЕРЕХОД

«Теперь есть что сравнивать: оба кластера живы, а self-managed ещё и прошёл обслуживание с upgrade».

Время: 214:00–228:00

ЭКРАН: доска с итоговой таблицей → split terminal/browser.

ГОВОРЮ

«Сравниваем по шести наблюдаемым признакам.

Первое — ноды. В self-managed видны control-plane и workers. В managed — только worker-ноды.

Второе — etcd. В self-managed его Pod'ы и backup — наша ответственность. В managed control plane скрыт.

Третье — сертификаты. На self-managed я сам смотрю срок и планирую ротацию. Managed обслуживает control-plane-сертификаты, но это не отменяет наших secrets и app certificates.

Четвёртое — внешний Service. В managed облачный контроллер получает адрес у провайдера. В self-managed сейчас увидим `pending` и установим MetalLB.

Пятое — обновление. Managed даёт API/операцию провайдера; self-managed требует playbook, sequencing и доказательства.

Шестое — цена ответственности. Managed оплачивается деньгами и ограничениями платформы. Self-managed даёт больше контроля, но требует времени, on-call, backup, patching и capacity».

📄 V2-C1-S16-C01 · Сравнить владение control plane, etcd и внешним IP

```
kubectl --context yc-managed apply -k "$REPO_ROOT/kubernetes/devops-may-cry/base"
kubectl --context yc-managed -n traffic-lab rollout status deploy/devops-may-cry
--timeout=180s
kubectl --context yc-managed -n traffic-lab patch svc devops-may-cry \
  --type merge -p '{"spec":{"type":"LoadBalancer"}}'
kubectl --context kubespray get nodes
kubectl --context yc-managed get nodes
kubectl --context kubespray -n kube-system get pods | grep etcd
kubectl --context yc-managed -n kube-system get pods | grep etcd || true
kubectl --context yc-managed -n traffic-lab wait \
  --for=jsonpath='{.status.loadBalancer.ingress[0].ip}' \
  service/devops-may-cry --timeout=300s
kubectl --context yc-managed -n traffic-lab get svc devops-may-cry -o wide
```

✅ После команды

Ожидая: self-managed показывает control-plane/etcd, managed скрывает их; облачный Service получает внешний адрес от провайдера.

🗣️ ГОВОРЮ

«И ещё доступ. У managed обычно короткоживущий токен через cloud identity. У self-managed часто начинают с клиентского сертификата; его утечка равна утечке доступа, поэтому kubeconfig защищаем как секрет. В зрелой схеме и self-managed может использовать OIDC или другой централизованный вход».

V2-C1-S16-C02 · Сравнить kubernetes и ответственность

```
kubectl config get-contexts
kubectl config view --context yc-managed --minify -o yaml
kubectl config view --context kubespray --minify -o yaml
stat -f '%Sp %N' "$HOME/.kube/config"
yc managed-kubernetes cluster list --format json \
  | jq -r '.[.] | [.name, .status] | @tsv'
yc managed-kubernetes node-group list --format json \
  | jq -r '.[.] | [.name, .status] | @tsv'
```

После команды

Ожидая: два разных способа доступа; содержимое сертификатов/токенов в кадре не раскрываем. Managed снимает часть эксплуатации платформы, но не отвечает за workload.

ФОРМУЛИРОВКА

Не говорить «managed лучше» или «self-managed лучше». Вопрос — какую часть эксплуатации команда готова купить у провайдера и какую обязана контролировать сама.

ПЕРЕХОД

«Один пункт таблицы пока красный: LoadBalancer в self-managed. Чиним его руками и сразу проверяем границу L2 в облаке».

Время: 228:00–251:00

ЭКРАН: доска «L2 vs L3 в MetalLB» → VS Code → терминал.

ГОВОРЮ

«Service типа LoadBalancer — это запрос на внешний адрес. В managed его обрабатывает интеграция облака. В обычном self-managed кластере обработчика нет, поэтому адрес остаётся `pending`».

V2-C1-S17-C01 · До MetalLB: Service LoadBalancer остаётся pending

```
cd "$REPO_ROOT/01_ЧАСТЬ_1_КЛАСТЕР/11_1.9_metallb"
kubectl --context kubespray apply -k ./devops-may-cry-base
kubectl --context kubespray -n traffic-lab rollout status deploy/devops-may-cry -
-timeout=180s
kubectl --context kubespray apply -f ./devops-may-cry-loadbalancer.yaml
kubectl --context kubespray -n traffic-lab get svc devops-may-cry-metallb
```

После команды

Ожидая: `EXTERNAL-IP <pending>`. В self-managed кластере нет cloud-controller, который сам выделит адрес.

ГОВОРЮ

«Для L2 режима MetalLB нужен реальный свободный pool в той же сети, где соседи могут получить ARP-анонс. Pool нельзя выбирать по принципу «ping молчит»: адрес может быть зарезервирован DHCP или другим IPAM.

Kube-proxy в IPVS-режиме требует strictARP. Мы включили его до установки кластера и сейчас проверяем live config».

V2-C1-S17-C02 · Проверить pool и strictARP до установки

```
source "$REPO_ROOT/recording/.recording.env"
: "${NODE_SUBNET_CIDR:?fill recording/.recording.env}"
: "${METALLB_POOL_START:?fill recording/.recording.env}"
: "${METALLB_POOL_END:?fill recording/.recording.env}"
printf 'node subnet: %s\nreserved pool: %s-%s\n' \
    "$NODE_SUBNET_CIDR" "$METALLB_POOL_START" "$METALLB_POOL_END"
kubectl --context kubespray get nodes -o wide
kubectl --context kubespray -n kube-system get configmap kube-proxy -o yaml |
grep -A3 -B3 strictARP
```

После команды

Ожидая: pool входит в node subnet, зарезервирован вне DHCP/IPAM, `strictARP: true`. Ping не доказывает, что IP свободен.

ГОВОРЮ

«Устанавливаю chart версии `0.16.1` напрямую. Затем применяю два CR: `IPAddressPool` и `L2Advertisement`. Speaker на нодах объявляет выбранный адрес в L2-домене».

ПОКАЗАТЬ

- `metallb-demo.yaml` с placeholders.
- `IPAddressPool`.
- `L2Advertisement`.
- В BGP reference: `BGPPeer` и `BGPAdvertisement`, но не применять.

V2-C1-S17-C03 · Установить MetalLB L2 и получить EXTERNAL-IP

```
helm repo add metallb https://metallb.github.io/metallb
helm repo update metallb
helm upgrade --install metallb metallb/metallb \
  --namespace metallb-system --create-namespace \
  --version 0.16.1 --wait --timeout 5m
METALLB_CONFIG="$(mktemp "${TMPDIR:-/tmp}/video2-metallb.XXXXXX.yaml")"
sed -e "s/__POOL_START__/$METALLB_POOL_START/g" \
  -e "s/__POOL_END__/$METALLB_POOL_END/g" \
  ./metallb-demo.yaml > "$METALLB_CONFIG"
cat "$METALLB_CONFIG"
kubectl --context kubespray apply -f "$METALLB_CONFIG"
kubectl --context kubespray -n metallb-system get pods -o wide
kubectl --context kubespray -n metallb-system get ipaddresspool,l2advertisement -o wide
kubectl --context kubespray -n traffic-lab wait \
  --for=jsonpath='{.status.loadBalancer.ingress[0].ip}' \
  service/devops-may-cry-metallb --timeout=120s
kubectl --context kubespray -n traffic-lab get svc devops-may-cry-metallb -o wide
kubectl --context kubespray -n metallb-system logs \
  -l app.kubernetes.io/component=speaker --tail=40
```

После команды

Ожидаю: controller/speaker Ready и Service получает адрес из pool. Это доказывает распределение адреса, но ещё не внешнюю достижимость.

ГОВОРЮ

«Service получил EXTERNAL-IP. Это доказательство назначения адреса, но не доказательство, что клиент снаружи до него дошёл.

В физической сети или разрешённом L2-сегменте сосед увидит ARP и curl пройдёт. Многие cloud VPC фильтруют чужие ARP-анонсы или не дают VM владеть произвольным адресом. Тогда статус заполнен, а трафика нет. Это не полурбочий LoadBalancer — это неподходящая модель сети.

L3/BGP решает задачу иначе: MetalLB анонсирует маршруты настоящему router peer. Без маршрутизатора и настроенного peering применять BGP YAML бессмысленно, поэтому показываю его как reference, а не разыгрываю успех».

V2-C1-S17-C04 · Проверить границу L2 в облачной сети и показать BGP reference

```
METALLB_IP="$(kubectl --context kubespray -n traffic-lab get service devops-may-cry-metallb -o jsonpath='{.status.loadBalancer.ingress[0].ip}')"
INV="$REPO_ROOT/kubespray/inventory/video2/inventory.ini"
SSH_KEY="${VIDE02_SSH_KEY:-$HOME/.ssh/id_ed25519}"
source "$REPO_ROOT/kubespray/.venv/bin/activate"
ansible -i "$INV" node1 --private-key "$SSH_KEY" -m ansible.builtin.shell \
    -a "ip neigh show $METALLB_IP; curl --max-time 5 -sv http://$METALLB_IP/readyz;
rc=\$?; printf 'curl_rc=%s\n' \"\$rc\"; exit 0"
sed -n '1,220p' ./metallb-bgp-reference.yaml
rm -f "$METALLB_CONFIG"
```

После команды

Ожидая одно из двух: HTTP 200 и `curl_rc=0` в разрешённом L2-домене либо контролируемый timeout с ненулевым `curl_rc` в cloud VPC, которая не пропускает нужный ARP/L2. Ansible-задача остаётся зелёной, потому что обе ветки здесь являются измерением. BGP-файл не применяем без реального peer/router.

L2 И BGP КОРОТКО

- L2: один speaker отвечает за IP через ARP/NDP; проще, но ограничен broadcast domain.

- BGP: ноды анонсируют prefix маршрутизатору; сложнее, требует сетевой инфраструктуры, зато лучше масштабируется и может использовать ECMP.
- В облаке часто берут native LB. В on-prem/bare metal MetalLB может быть нормальным решением, если сетевая модель это поддерживает.

ПЕРЕХОД

«MetalLB живёт внутри Kubernetes и не заменяет отдельный HA-этаж перед кластером. Теперь покажу внешний вход на двух Linux-балансировщиках».

Время: 251:00–282:00

ЭКРАН: доска «Внешняя точка входа» → `ansible/external-ha` → два терминала.

ГОВОРЮ

«Схема другая: клиент обращается к VIP, keepalived переносит его между lb1 и lb2, HAProxy в L4 режиме отправляет TCP на NodePort каждой Kubernetes-ноды, а дальше работает kube-proxy и Service.

В этой лаборатории VIP `10.77.0.10` живёт внутри VXLAN между двумя LB. Это позволяет воспроизвести VRRP-подобный failover в облаке, но не превращает VIP в публичный cloud IP. Floating addresses нужны только для SSH.

Сначала публикую наше приложение NodePort `30080` и проверяю EndpointSlice. Потом читаю committed inventory example и graph без hostvars; реальный inventory с floating-адресами на экран не вывожу. Ansible сам снимает private IP facts, их не переписываем с панели».

📄 V2-C1-S18-C01 · Подготовить NodePort и прочитать HA inventory

```
cd "$REPO_ROOT"
HA_DIR="$REPO_ROOT/ansible/external-ha"
INV="$HA_DIR/inventory.ini"
SSH_KEY="{VIDE02_SSH_KEY:-$HOME/.ssh/id_ed25519}"
source "$REPO_ROOT/recording/.recording.env"
source "$REPO_ROOT/kubespray/.venv/bin/activate"
kubectl --context kubespray apply -f "$REPO_ROOT/kubernetes/devops-may-cry/services/nodeport.yaml"
kubectl --context kubespray -n traffic-lab get service devops-may-cry-ha-nodeport
kubectl --context kubespray -n traffic-lab get endpointslice \
  -l kubernetes.io/service-name=devops-may-cry-ha-nodeport -o wide
sed -n '1,180p' "$HA_DIR/inventory.example.ini"
ansible-inventory -i "$INV" --graph
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.ping
ansible -i "$INV" all --private-key "$SSH_KEY" -m ansible.builtin.setup \
  -a 'filter=ansible_default_ipv4'
```

✅ После команды

Ожидаю: NodePort `30080`, ready endpoints, безопасный example и graph с пятью host names без hostvars. Реальный inventory адреса не печатает; floating IP нужен только для SSH, private facts используются для VXLAN/backends.

🗣️ ГОВОРЮ

«Открываю `site.yml`. Первый play собирает facts со всех пяти VM. Роль `vxlan_l2` создаёт общий лабораторный L2 для lb1/lb2. Роль `haproxy` ставит L4 frontend и три private NodePort backend. Роль `keepalived` следит за HAProxy и назначает VIP только живому балансировщику.

В шаблоне HAProxy `mode tcp`. Проверка backend — HTTP `GET /readyz`, но dataplane остаётся L4. В keepalived видны `interval/fall`, `advert_int` и `nopreempt`.

Сначала syntax check, затем отдельный preflight с lb1 до каждого private backend. Если backend недоступен, основной playbook не запускаю».

☰ V2-C1-S18-C02 · Прочитать роли и развернуть HAProxy/keepalived

```
sed -n '1,240p' "$HA_DIR/site.yml"
sed -n '1,220p' "$HA_DIR/roles/haproxy/templates/haproxy.cfg.j2"
sed -n '1,220p' "$HA_DIR/roles/keepalived/templates/keepalived.conf.j2"
ansible-playbook -i "$INV" "$HA_DIR/site.yml" --syntax-check --private-key "$SSH_KEY"
ansible-playbook -i "$INV" "$HA_DIR/preflight-backends.yml" --private-key "$SSH_KEY"
ansible-playbook -i "$INV" "$HA_DIR/site.yml" --private-key "$SSH_KEY"
```

✅ После команды

Ожидая: preflight достигает всех private NodePort backends; затем `PLAY RECAP` без failed. HAProxy работает в L4 `mode tcp`, health check идёт на `/readyz`.

🗣 ГОВОРЮ

«До отказа нужны три доказательства: оба сервиса active, VIP ровно на одном LB и HTTP 200 через VIP».

V2-C1-S18-C03 · Доказать VIP и HTTP до отказа

```
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl is-active haproxy keepalived'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100'
VIP_OWNERS=()
for host in lb1 lb2; do
  if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
    | grep -q "$HA_VIP/24"; then
    VIP_OWNERS+=("$host")
  fi
done
test "${#VIP_OWNERS[@]}" -eq 1
printf 'VIP owner: %s\n' "${VIP_OWNERS[0]}"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become -m ansible.builtin.uri \
  -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"
```

После команды

Ожидая: оба сервиса active, VIP `10.77.0.10` находится ровно на одном LB, HTTP 200. Это лабораторный VIP внутри VXLAN-пары, не публичный cloud IP.

ГОВОРЮ

«Активный LB не зашит в сценарий: определяю владельца VIP по факту. На lb1 запускаю фоновую HTTP-пробу раз в секунду. Затем останавливаю только HAProxy на активном балансировщике.

Track script замечает отказ, keepalived снимает VIP, второй LB становится владельцем. Bounded-циклом жду сразу три условия: владелец снова ровно один, это уже другой LB и HTTP вернулся к 200. Затем показываю journal и probe log. Длительность

провала читаем с пробы; не обещаем красивую цифру заранее».

📄 V2-C1-S18-C04 · Найти активный LB и показать failover

```
VIP_OWNERS=()
for host in lb1 lb2; do
    if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
        -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' | grep -q
"$HA_VIP/24"; then
        VIP_OWNERS+=("$host")
    fi
done
test "${#VIP_OWNERS[@]}" -eq 1
ACTIVE_LB="${VIP_OWNERS[0]}"
printf 'VIP owner before failure: %s\n' "$ACTIVE_LB"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become -m ansible.builtin.shell \
    -a "rm -f /tmp/vip-probe.log; nohup sh -c 'for i in \$(seq 1 25); do printf
\"%s \" \"\$(date +%T)\"; curl --max-time 2 -sS -o /dev/null -w \"%
{http_code}\\n\" http://$HA_VIP:$HA_PORT/readyz || echo timeout; sleep 1; done'
>/tmp/vip-probe.log 2>&1 </dev/null &"
ansible -i "$INV" "$ACTIVE_LB" --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'systemctl stop haproxy'
NEW_OWNER=''
for _ in $(seq 1 30); do
    VIP_OWNERS=()
    for host in lb1 lb2; do
        if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
            -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
            | grep -q "$HA_VIP/24"; then
            VIP_OWNERS+=("$host")
        fi
    done
    if [ "${#VIP_OWNERS[@]}" -eq 1 ] \
        && [ "${VIP_OWNERS[0]}" != "$ACTIVE_LB" ] \
        && ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
```

```
-m ansible.builtin.uri \
-a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200" \
>/dev/null 2>&1; then
NEW_OWNER="${VIP_OWNERS[0]}"
break
fi
sleep 1
done
test -n "$NEW_OWNER"
printf 'VIP owner after failure: %s\n' "$NEW_OWNER"
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
-m ansible.builtin.command -a 'ip -4 addr show dev vxlan100'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become -m
ansible.builtin.shell \
-a "journalctl -u keepalived --since '3 minutes ago' --no-pager | tail -24"
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become \
-m ansible.builtin.command -a 'cat /tmp/vip-probe.log'
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become -m ansible.builtin.uri \
-a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"
```

✅ После команды

Ожидая: VIP переезжает на другой LB, в keepalived есть смена состояния, в probe log виден фактический разрыв и последующие 200. Не обещаем `<1s`.

🗣 ГОВОРЮ

«Возвращаю HAProxy. Из-за `nopreempt` восстановленный узел не отбирает VIP автоматически. Это нормально: важно, что оба сервиса active, один владелец VIP и HTTP 200».

V2-C1-S18-C05 · Restore и итоговая проверка

```
ansible -i "$INV" "$ACTIVE_LB" --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl start haproxy'
ansible -i "$INV" load_balancers --private-key "$SSH_KEY" --become \
  -m ansible.builtin.command -a 'systemctl is-active haproxy keepalived'
VIP_OWNERS=()
for host in lb1 lb2; do
  if ansible -i "$INV" "$host" --private-key "$SSH_KEY" --become \
    -m ansible.builtin.command -a 'ip -4 addr show dev vxlan100' \
    | grep -q "$HA_VIP/24"; then
    VIP_OWNERS+=("$host")
  fi
done
test "${#VIP_OWNERS[@]}" -eq 1
ansible -i "$INV" lb1 --private-key "$SSH_KEY" --become -m ansible.builtin.uri \
  -a "url=http://$HA_VIP:$HA_PORT/readyz status_code=200"
```

После команды

Ожидая: оба сервиса active, владелец VIP по-прежнему ровно один и HTTP 200. Из-за `no-preempt` VIP после восстановления не обязан вернуться на lb1.

В ПРОДЕ

Здесь обязательно добавились бы реальный L2/BGP/cloud-failover механизм, firewall, observability, config delivery, secret management, несколько failure modes и регулярный failover test. DNS может переключать между endpoints, но TTL, caches и время реакции делают его другим механизмом, не мгновенной заменой VIP.

ЕСЛИ FAILOVER НЕ ПРОШЁЛ

Остановить дубль. Проверить facts/private IP, UDP 4789 для лабораторного VXLAN, интерфейс `vxlan100`, keepalived journal, HAProxy status и доступность NodePort с lb1. Не трогать системный VPN.

ПЕРЕХОД

«Теперь у нас два живых кластера, настоящий upgrade, MetalLB и отдельная отказоустойчивая точка входа. Соберу выводы».

Время: 282:00–291:00

ЭКРАН: итоговая таблица → карта следующих частей → DEVOPS MAY CRY.

ГОВОРЮ

«Начинали с двух файлов Terraform и локального Ansible. Рабочие пять VM получили из урока, такой же состав стенда описали в Terraform, а затем собрали Kubespray-кластер, managed-кластер и отдельную пару балансировщиков.

Главная цепочка такая. Terraform создаёт инфраструктуру и отдаёт outputs. Inventory назначает машинам роли. Ansible приводит Linux к нужному состоянию. Kubespray этими же механизмами ставит Kubernetes. После установки мы доказываем API, CNI, DNS и HTTP, а не останавливаемся на `get nodes`.

Managed и self-managed отличаются не наличием YAML и не качеством приложений. Отличается граница эксплуатационной ответственности.

MetalLB закрывает Service LoadBalancer внутри подходящей сетевой модели. Внешний HA-этаж решает другую задачу: стабильная точка входа и failover пары балансировщиков.

В следующей части DEVOPS MAY CRY перестанет быть просто тестовым Deployment. Разберём, как scheduler выбирает ноду, выделим место для Postgres, пройдем taints, affinity, topology spread, quota и Pending — всё на этом же приложении».

ПОКАЗАТЬ

- Список пяти VM и два Kubernetes context.
- Коротко `kubectl get nodes` на обоих.
- Финальная схема: Terraform → inventory → Kubespray → cluster → Service → external HA.

- Карта части 2 без обещания «через минуту», если запись будет отдельной.

ПОСЛЕ REC

📄 V2-C1-S19-C01 · Cleanup только после остановки записи

```
kubectl --context kubespray -n traffic-lab delete service devops-may-cry-metallb
--ignore-not-found
helm uninstall metallb -n metallb-system || true
kubectl --context kubespray delete namespace metallb-system --ignore-not-found
cd "$REPO_ROOT/infra/managed/terraform"
terraform destroy
cd "$REPO_ROOT/infra/self-managed/terraform"
terraform destroy
```

✅ После команды

Ожидая: cleanup выполняется только после REC и после сохранения доказательств. Перед каждым destroy читаем plan; state и inventory не показываем в кадре.

⚠️ ВАЖНО

Cleanup не читать и не выполнять в основном дубле. Если части 2–4 пишутся на тех же стендах, ничего не уничтожать. Сначала сохранить запись, логи и нужные доказательства.

- Хороший дубль: вслух «ИЗУМРУД».
- Долгое ожидание: «АНТРАКТ», после возврата «РУБЕЖ».
- Ошибка: «БРАК-БРАК», остановить, восстановить состояние, новый дубль начать с полного переходного предложения.
- Не импровизировать ожидаемый вывод. Сверить команду по ID и выполнить reset вне REC.
- Не использовать README как запасной суфлёр: если в этой колоде не объяснён файл или результат, это баг материала.